内 容 安 排

章	内 容	学时	章	内 容	学时
1	绪 论	2	7	图	5
2	线性表	5	8	动态存储管理	略
3	栈和队列	3	9	查找	3
4	串	3	10	内部排序	5
5	数组和广义表	3	11	外部排序	略
6	树和二叉树	7	12	文件	略

数据结构课程的内容

- 数据结构
  - 逻辑结构
    - 线性结构（线性表、栈　、队、串、数组）
    - 非线性结构
      - 树结构
      - 图结构
  - 物理（存储）结构
    - 顺序结构
    - 链式结构
    - 索引结构
    - 散列结构
  - 数据运算
    - 插入运算
    - 删除运算
    - 修改运算
    - 查找运算
    - 排序运算

第9章　查找

9.1 基本概念

9.2 静态查找表

9.3 动态查找表

9.4 哈希表

教材第8、11和12章省略，因《操作系统》课程会涉及。

9.1 基本概念

是一种数据结构

查找表 ——由同一类型的数据元素（或记录）构成的集合。

查 找 ——查询(Searching)特定元素是否在表中。

查找成功 ——若表中存在特定元素，称查找成功，应输出该记录；

查找不成功 ——否则，称查找不成功（也应输出失败标志或失败位置）

静态查找 ——只查找，不改变集合内的数据元素。

动态查找 ——既查找，又改变（增减）集合内的数据元素。

关键字 ——记录中某个数据项的值，可用来识别一个记录

（ 预先确定的记录的某种标志）

主关键字 ——可以唯一标识一个记录的关键字

例如“学号”

次关键字 ——识别若干记录的关键字

例如“女”

讨论：

讨论1：查找的过程是怎样的？

给定一个值K，在含有n个记录的文件中进行搜索，寻找一个关键字值等于K的记录，如找到则输出该记录，否则输出查找不成功的信息。

讨论2：对查找表常用的操作有哪些？

“特定的”=关键字

- 查询某个“特定的”数据元素是否在表中；
- 查询某个“特定的”数据元素的各种属性；
- 在查找表中插入一元素；
- 从查找表中删除一元素。

讨论3：有哪些查找方法？

例如查字典

查找方法取决于表中数据的排列方式；

针对静态查找表和动态查找表的查找方法也有所不同。

讨论4：如何评估查找方法的优劣？

明确：**查找的过程就是将给定的K值与文件中各记录的关键字项进行比较的过程。所以用比较次数的平均值来评估算法的优劣。称为平均查找长度ASL。**

Average Search Length

$$ASL=\sum P_i \cdot C_i$$

统计意义上的数学期望值

其中：

n是文件记录个数；

P_i是查找第i个记录的查找概率（通常取等概率,即$P_i=1/n$）；

C_i是找到第i个记录时所经历的比较次数。

物理意义：**假设每一元素被查找的概率相同，则查找每一元素所需的比较次数之总和再取平均，即为ASL。**

显然，ASL值越小，时间效率越高

9.2 静态查找表

针对静态查找表的查找算法主要有：

一、顺序查找（线性查找）

二、折半查找（二分或对分查找）

三、分块查找（索引顺序查找）

四、静态树表的查找

静态查找表的定义
（抽象数据类型）
参见教材P216

```
ADT StaticSearchTable {
数据对象D:
数据关系R:
基本操作P:
} ADT StaticSearchTable
```

一、顺序查找（Linear search，又称线性查找）

顺序查找：用逐一比较的办法顺序查找关键字，这显然是最直接的办法。

- 对顺序结构如何线性查找?
- 对单链表结构如何线性查找?
- 对非线性树结构如何顺序查找?

见下页之例或教材P216

从头指针始 “顺藤摸瓜”

借助各种遍历操作!

（1）顺序表的机内存储结构：

```
typedef struct {
        ElemType   *elem;    //表基址，0号单元留空。表容量为全部元素
            int        length;   //表长，即表中数据元素个数
}SSTable;
```

（2）算法的实现：

技巧：把待查关键字key存入表头或表尾（俗称“哨兵”），这样可以加快执行速度。

例： 若将待查找的特定值key存入顺序表的首部（如0号单元），并从后向前逐个比较,就不必担心“查不到”。

```
int Search_Seq( SSTable  ST , KeyType  key ){

ST.elem[0

for( i=ST.le

    return i

} // Sea
```

//在顺序表ST中，查找关键字与key相同的元素；若成功，返回其位置信息，否则返回0

//设立哨兵，可免去查找过程中每一步都要检测是否查找完毕。当n>1000时，查找时间将减少一半

//不要用for(i=1; i<=n; i++)
或 for(i=n; i>0; --i)

//找不到=
找得到=成功，返回值i正好代表所找元素的位置。

讨论1：查找失败怎么办？

——返回特殊标志，例如返回空记录或空指针。前例中设立了“哨兵”，就是将关键字送入末地址ST.elem[0].key使之结束，并返回 i=0。

讨论2：查找效率怎样计算？

——用平均查找长度ASL衡量。

讨论3：如何计算ASL？

分析：

查找第1个元素所需的比较次数为1；

查找第2个元素所需的比较次数为2；

……

查找第n个元素所需的比较次数为n；

未考虑查找不成功的情况：查找哨兵所需的比较次数为n+1

总计全部比较次数为：1＋2＋…＋n = (1+n)n/2

若求某一个元素的平均查找次数，还应当除以n（等概率），

即： ASL＝（1＋n）/2 ，时间效率为 O(n)

这是查找成功的情况

讨论4：顺序查找的特点：

优点：算法简单，且对顺序结构或链表结构均适用。

缺点： ASL 太大，时间效率太低。

如何改进?

二、折半查找（又称二分查找或对分查找）

这是一种容易想到的查找方法。

先给数据排序（例如按升序排好），形成有序表，然后再将key与正中元素相比，若key小，则缩小至右半部内查找；再取其中值比较，每次缩小1/2的范围，直到查找成功或失败为止。

- 对顺序表结构如何编程实现折半查找算法？
  ——**见下页之例，或见教材（P219）**
- 对单链表结构如何折半查找？
  ——**无法实现！因全部元素的定位只能从头指针head开始**
- 对非线性(树)结构如何折半查找？
  ——**可借助二叉排序树来查找（属动态查找表形式）。**

折半查找举例： 已知如下11个元素的有序表：

（05 13 19 21 37 56 64 75 80 88 92），请查找关键字为21和85的数据元素。

Low指向待查元素所在区间的下界

mid指向待查元素所在区间的中间位置

high指向待查元素所在区间的上界

解：① 先设定3个辅助标志: low,high,mid,

显然有：mid=⌊(low+high)/2⌋

② 运算步骤：

(1) low =1,high =11 ,故mid =6 ，待查范围是 [1,11];

(2) 若 key < ST.elem[mid].key ，说明 key∈[mid+1,high] ，

则令：low =mid+1;重算 mid= ⌊(low+high)/2⌋; .

(3) 若 key > ST.elem[mid].key ，说明key∈[low ,mid-1],

则令：high =mid-1;重算 mid ;

(4)若 ST.elem[mid].key = key，说明查找成功，元素序号=mid;

结束条件： （1）查找成功 ： ST.elem[mid].key = key

（2）查找不成功 ： high<low （意即区间长度小于0)

此例可作为上机内容

讨论1：若关键字不在表中，怎样得知并及时停止查找？

——典型标志是：当查找范围的上界≤下界时停止查找。

讨论2：如何计算二分查找的时间效率（ASL）？

推导过程

经1次比较就查找成功的元素有1个（2^0），即中间值；

经2次比较就查找成功的元素有2个（2^1），即1/4处和3/4处；

3次比较就查找成功的元素有4个（2^2），即1/8，3/8， 5/8，7/8处

4次比较就查找成功的元素有8个（2^3），即1/16处，3/16处……

……

则第m次比较时查找成功的元素应该有（2^{m-1}）个。

注：为方便起见，假设表中全部n个元素刚好是 2^m-1个，此时暂不讨论第m次比较后还有剩余元素的情况。

全部比较总次数为$1\times2^0+2\times2^1+3\times2^2+4\times2^3\ldots+m\times2^{m-1}$

$$=\sum_{j=1}^{m} j\cdot 2^{j-1}$$

请注意：ASL的含义是“平均每个数据的查找时间”，而前式是n个数据查找时间的总和，所以：

$$ASL = \frac{1}{n}\sum_{j=1}^{m} j \cdot 2^{j-1} = \frac{n+1}{n}\log_2(n+1) - 1 \approx \log_2 n$$

（详细推导过程见教材P221的附录1）

课堂练习1（多选题）：使用折半查找算法时，要求被查文件：

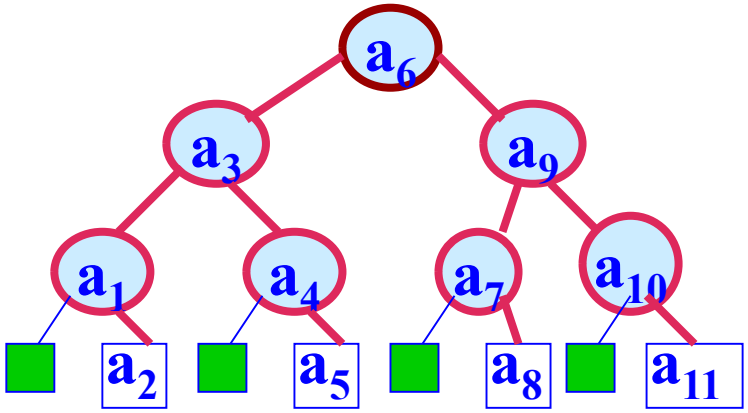
A．采用链式存贮结构　B．记录的长度≤12

C．采用顺序存贮结构　D．记录按关键字递增有序

课堂练习2： 在有序线性表a[20]上进行折半查找，则平均查找长度为____3.7____。

再用查找二叉树/判定树来分析ASL（参见教材P220）

以（$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}$）为例：



判定树由表中元素个数决定，且n个元素表的折半查找判定树是唯一的。

- 找到有序表中任一记录的过程就是：走了一条从根结点到与该记录对应结点的路径。
- 比较的关键字个数为：该结点在判定树上的层次数。
- 查找成功时比较的关键字个数最多不超过树的深度 d：

$$d = \lfloor \log_2 n \rfloor + 1$$

- 查找不成功的过程就是：走了一条从根结点到外部结点的路径。

讨论： 对顺序查找除了折半改进法外，还有无其他改进算法？
有，例如分块查找法。

三、分块查找（索引顺序查找）

思路：先让数据分块有序，即分成若干子表，要求每个子表中的数据元素值都比后一块中的数值小（但子表内部未必有序）。
然后将各子表中的最大关键字构成一个索引表，表中还要包含每个子表的起始地址（即头指针）。

例：

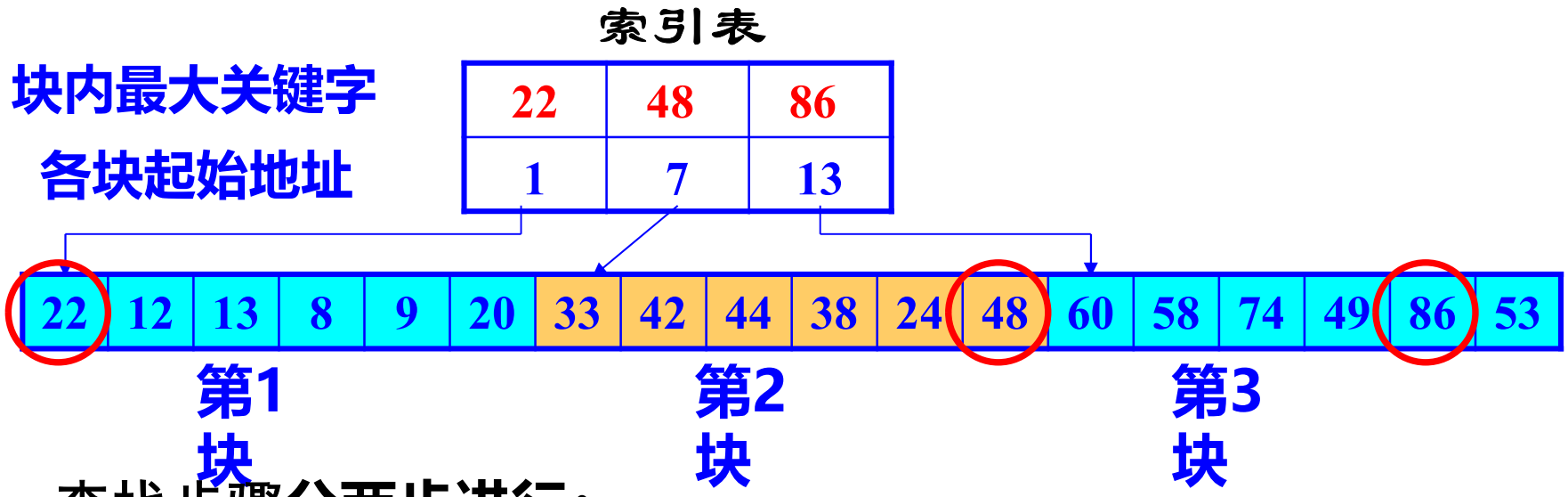
22	12	13	8	9	20	33	42	44	38	24	48	60	58	74	49	86	53

特点：块间有序，块内无序

分块查找过程举例：

特点：块间有序，块内无序

查找：块间折半，块内线性



查找步骤**分两步进行：**

① 对索引表使用折半查找法（因为索引表是有序表）；

② 确定了待查关键字所在的子表后，在子表内采用顺序查找法（因为各子表内部是无序表）；

查找效率ASL分析：

$$ASL = L_b + L_w$$

对索引表查找的ASL

对块内查找的ASL

$$ASL_{bs} \cong \log_2(\frac{n}{s}+1)+\frac{s}{2} \qquad (\log_2 n \le ASL_{bs} \le \frac{n+1}{2})$$

S为每块内部的记录个数，n/s即块的数目

例如当n=9，s=3时，

分块法的ASL_{bs} =3.5

而折半法的ASL≈$\log_2 n$=3.1

顺序法的ASL=(1＋n)/2 = 5

但折半法要预先全排序，仍需时间。

讨论：当有序表中各记录的查找概率不相等时，该如何查找？

提 醒：此时用折半查找法未必最优！（参见教材P222例）

改 进：**若将各记录的概率看作是二叉树之权值，则转为研究带权路径长度最小的二叉树（类似最优二叉树）。**

四、静态树表的查找

静态最优查找树算法——时间代价高；

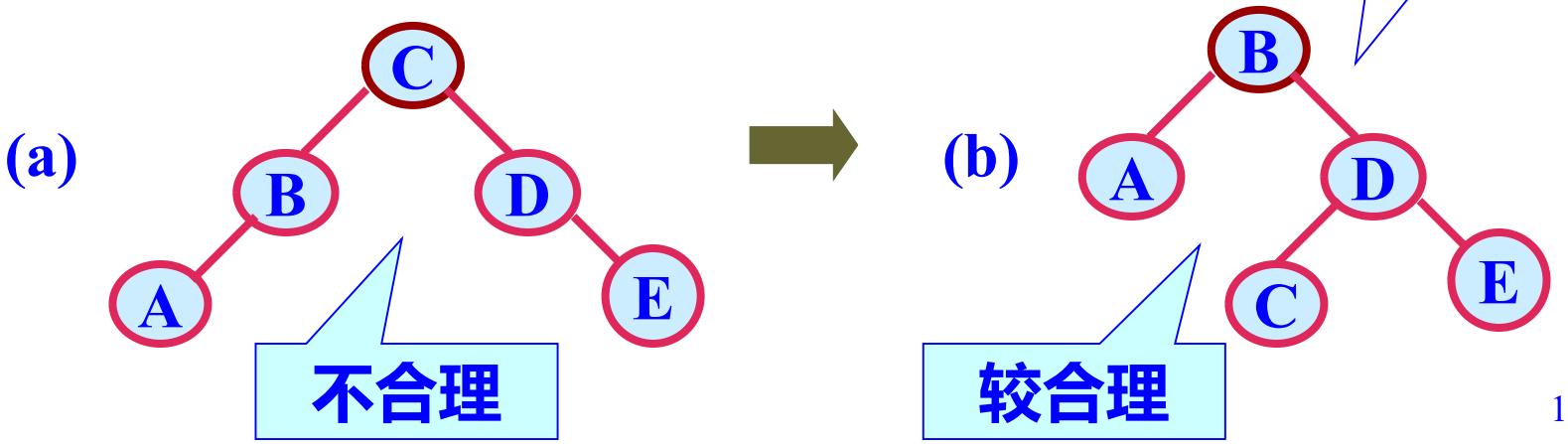
实用算法：次优查找树

算法详见教材P222—225

例：　有序表

对应权值

A	B	C	D	E
1	30	2	29	3



静态树表查找

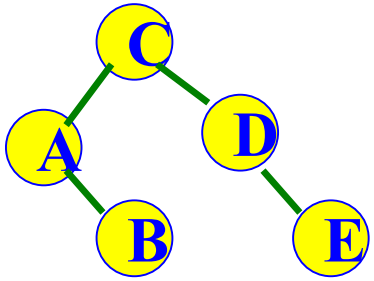
有序表的查找性能在等概率时,折半查找的判定树所描述查找过程其性能最优,但在各记录查找概率不等时,用折半查找判定树查找,其查找性能未必最优,例如:

有序表

A	B	C	D	E

查找概率 $P_1=0.1$ $P_2=0.2$ $P_3=0.1$ $P_4=0.4$ $P_5=0.2$

判定树

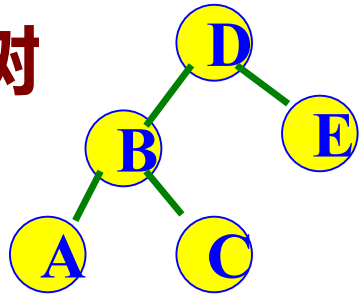


平均查找长度ASL

$$\sum_{i=1}^{n} PiCi = 0.1*2+0.2*3+0.1*1+0.4*2+0.2*3 = 2.3$$

若令给定值与第4个记录开始比较,则:

判定树



显然,当查找概率不等时,折半查找判定树并非最佳

平均查找长度ASL

$$\sum_{i=1}^{n} PiCi = 0.1*3+0.2*2+0.1*3$$
$$+0.4*1+0.2*2 = 1.8$$

❖如果只考虑查找成功的情况,查找性能最佳的判定树是其带权内路径长度之和PH值最小的二叉树

$$PH = \sum_{i=1}^{n} w_i h_i$$

h_i 为第i个结点在二叉树中的层次数,既比较次数

PH值最小的二叉树为静态最优查找树,但构造它的时间代价较高.

★一种构造PH值近似最小的次优查找树的方法:

已知一个按关键字有序的记录序列:

$$(r_l, r_{l+1}, \cdots r_{i-1}, r_i, r_{i+1}, \cdots, r_h)$$

其中

$$r_l.key < r_{l+1}.key < \cdots < r_h.key$$

构造方法是: 在以上序列中取第i ($l \le i \le h$)个记录构造根结点r_i,使得

$$\triangle P_i = \sum_{j=i+1}^{h} w_j - \sum_{j=l}^{i-1} w_j$$

取最小值($\triangle P_i = \mathrm{Min} \mid \triangle P_j \mid$ ($l \le j \le h$)) 然后分别将($r_l, r_{l+1}, \cdots r_{i-1}$)和($r_{i+1}, \cdots, r_h$)两个子序列用相同的方法构造两棵次优查找树作为根结点 r_i 的左右子树.

为便于计算$\triangle P_i$引入累计权值和:

$$sw_i = \sum_{j=l}^{i} w_j$$

设$w_{l-1}=0$ 和 $sw_{l-1}=0$ 则:

$$sw_{i-1} - sw_{1-1} = \sum_{j=l}^{i-1} w_j$$

$$sw_h - sw_i = \sum_{j=i+1}^{h} w_j$$

$$\triangle P_i = |(sw_h - sw_i) - (sw_{i-1} - sw_{l-1})|$$

$$= |(sw_h + sw_{l-1}) - sw_i - sw_{i-1}|$$

★通过例9-1 说明算法9.3 的执行过程

9.3 动态查找表

特点： **表结构在查找过程中动态生成。**

要求： **对于给定值key, 若表中存在其关键字等于key的记录，则查找成功返回；否则插入关键字等于key 的记录。**

典型的动态表——二叉排序树

一、二叉排序树的定义

二、二叉排序树的插入与删除

三、二叉排序树的查找分析

四、平衡二叉树

一、二叉排序树的定义

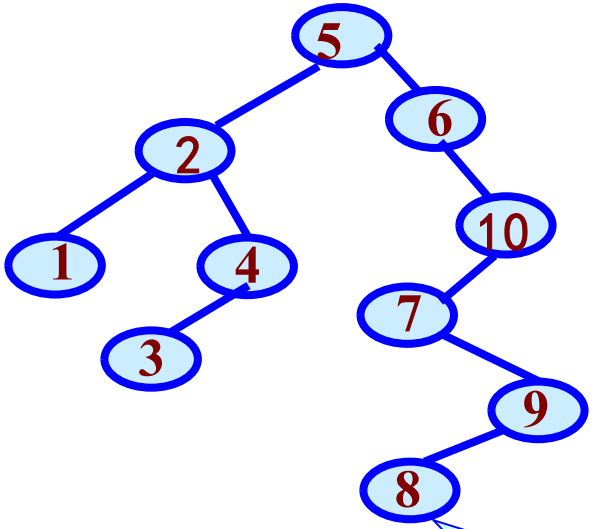
-----或是一棵空树；或者是具有如下性质的非空二叉树：

（1）左子树的所有结点均小于根的值；

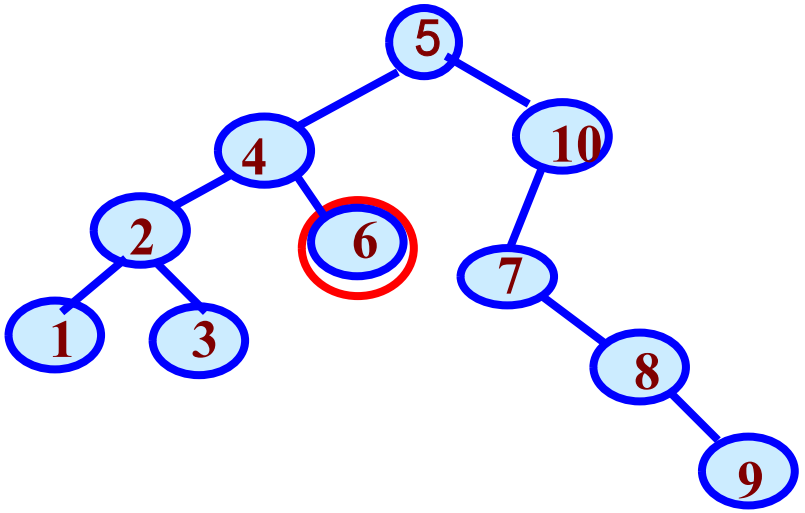
（2）右子树的所有结点均大于根的值；

（3）它的左右子树也分别为二叉排序树。

例：下列2种图形中，哪个不是二叉排序树？



（a）



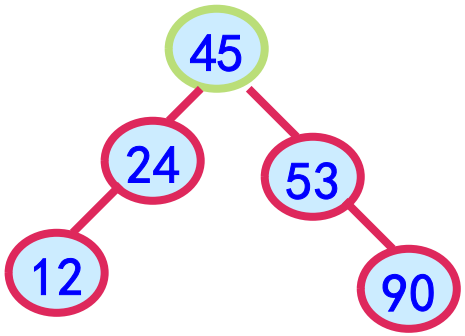
（b）

想一想：对它中序遍历之后是什么效果?

二、二叉排序树的插入与删除

例：输入待查找的关键字序列=（45，24，53，45，12，24，90）

查找不成功
则插入树中

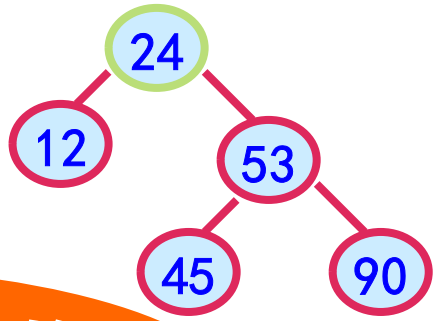


查找成功，返回

查找成功，返回

如果改变输入顺序为（24，53， 45，45，12，24，90），

则生成的二叉排序树形态不同



查找成功，返回

查找成功，返回

这种既查找又插入的过程称为动态查找

将线性表构造成二叉排序树的优点：

① 查找过程与顺序结构有序表中的折半查找相似，查找效率高；

② 中序遍历此二叉树，将会得到一个关键字的有序序列（即实现了排序运算）；

③ 如果查找不成功，能够方便地将被查元素插入到二叉树的叶子结点上，而且插入或删除时只需修改指针而不需移动元素。

二叉排序树既有类似于折半查找的特性，又采用了链表存储，它是动态查找表的一种适宜表示。

讨论1：二叉排序树的查找&插入算法如何实现？

思路：查找不成功，生成一个新结点s，插入到二叉排序树中；查找成功则返回。

程序：

```
SearchBST (K, &t) { //K为待查关键字，t为根结点指针
 p=t;      //p为查找过程中进行扫描的指针
 while（p!=NULL）{
 case {
 K= p->data: {查找成功，return }
 K< p->data : {q=p；p=p->L_child }  //继续向左搜索
 K> p->data : {q=p；p=p->R_child } //继续向右搜索
        }
 }  //查找不成功则插入到二叉排序树中
```

```
s =(BiTree)malloc(sizeof(BiTNode));
s->data=K; s ->L_child=NULL; s ->R_child=NULL;
//查找不成功，生成一个新结点s，插入到二叉排序树叶子处
case {
t=NULL:     t=s;   //若t为空，则插入的结点s作为根结点
K < q->data: q->L_child=s;  //若K比叶子小，挂左边
K > q->data: q->R_child=s; //若K比叶子大，挂右边
      }
return OK
}
```

以上为查找、插入二合一的非递归程序

查找模块（递归方式） 参见教材P228算法9.5（a）；

插入模块（非递归方式）参见教材P228算法9.6;

该模块就是“建树”的非递归算法！

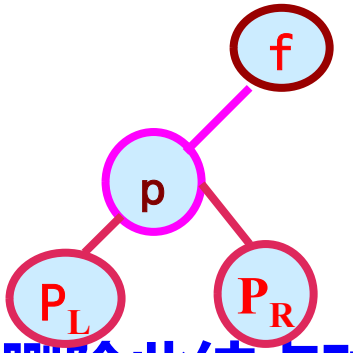
讨论2：二叉排序树的删除操作如何实现？

对于二叉排序树，删除树上一个结点相当于删除有序序列中的一个记录，要求删除后仍需保持二叉排序树的特性。

如何删除一个结点？

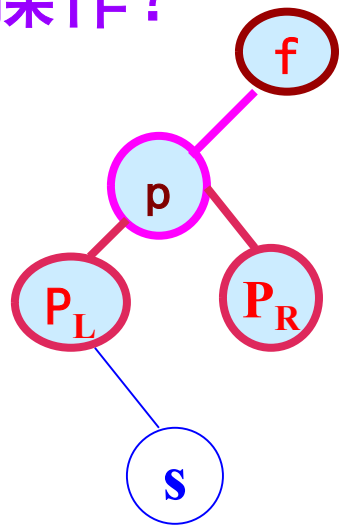
假设：*p表示被删结点的指针； P_L和P_R 分别表示*P的左、右孩子指针；

*f表示*p的双亲结点指针；并假定*p是*f的左孩子；则可能有三种情况：



- *p为叶子： **删除此结点时，直接修改*f指针域即可；**
- *p只有一棵子树（或左或右）： **令P_L或P_R为*f的左孩子即可；**
- *p有两棵子树：情况最复杂 →

难点：*p有两棵子树时，如何进行删除操作？



分析：

设删除前的中序遍历序列为：

…. P_L s p P_R f ….　　//显然p的直接前驱是s

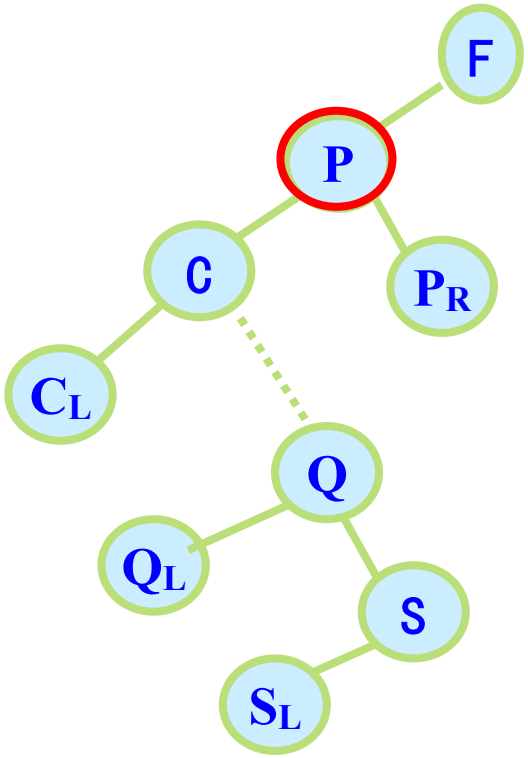
//s是*p左子树最右下方的结点

希望删除p后，其它元素的相对位置不变。有两种解决方法：

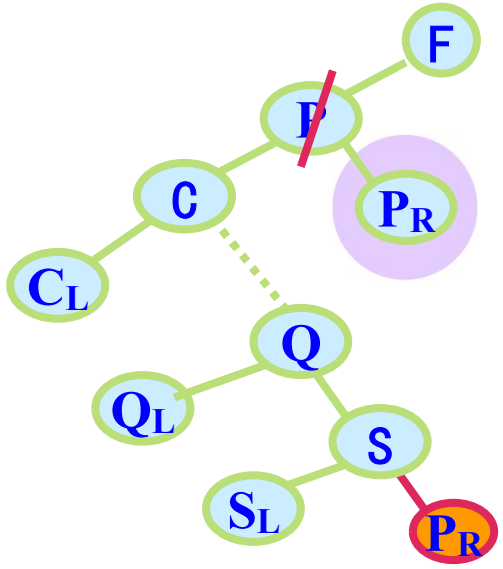
法1：令*p的左子树为 *f的左子树，*p的右子树接为*s的右子树；　//即$f_L=P_L$；$S_R=P_R$；

法2：直接令*s代替*p　// *s为*p左子树最右下方的结点

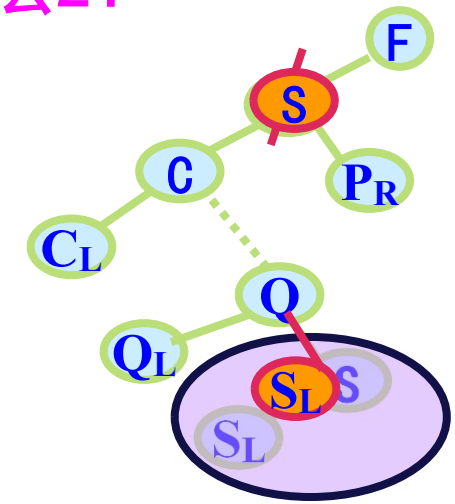
例：请从下面的二叉排序树中删除结点P。



法1：



法2：



三、二叉排序树的查找分析

最坏情况（单支树）：插入的n个元素从一开始就有序

此时树深为n ； ASL= (n+1)/2 ；与顺序查找情况相同。

最好情况（形态均衡）：与折半查找中的判定树相同

此时树深为：$\lfloor \log_2 n \rfloor + 1$ ；

$ASL = \log_2(n+1) - 1$ ；与折半查找情况相同。

一般情况： $ASL = 2(1+1/n) \ln n$ （与 log n 同阶）

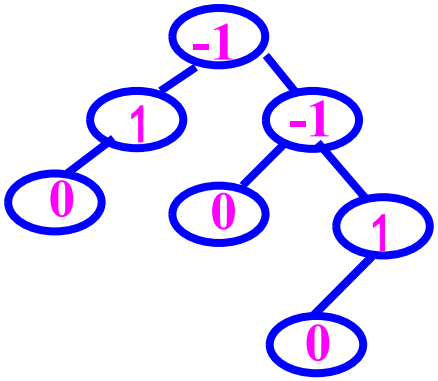
思考：如何提高二叉排序树的查找效率？

尽量让二叉树的形状均衡

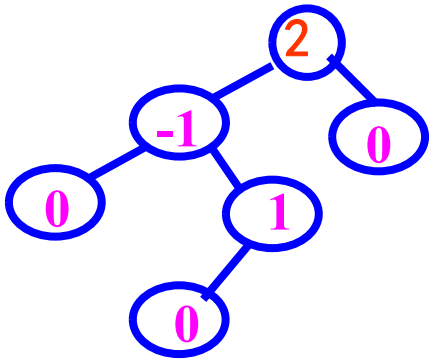
平衡二叉树

平衡二叉树的特点：

任一结点的平衡因子只能取：-1、0 或 1。



(a) 平衡树



(b) 不平衡树

对于一棵有n个结点的*AVL树*，
其高度保持在$O(\log_2 n)$数量级
则*ASL*也保持在$O(\log_2 n)$量级

如果在一棵AVL树中插入一个新结点，就有可能造成失衡，此时必须重新调整树的结构，使之恢复平衡。我们称调整平衡过程为平衡旋转。

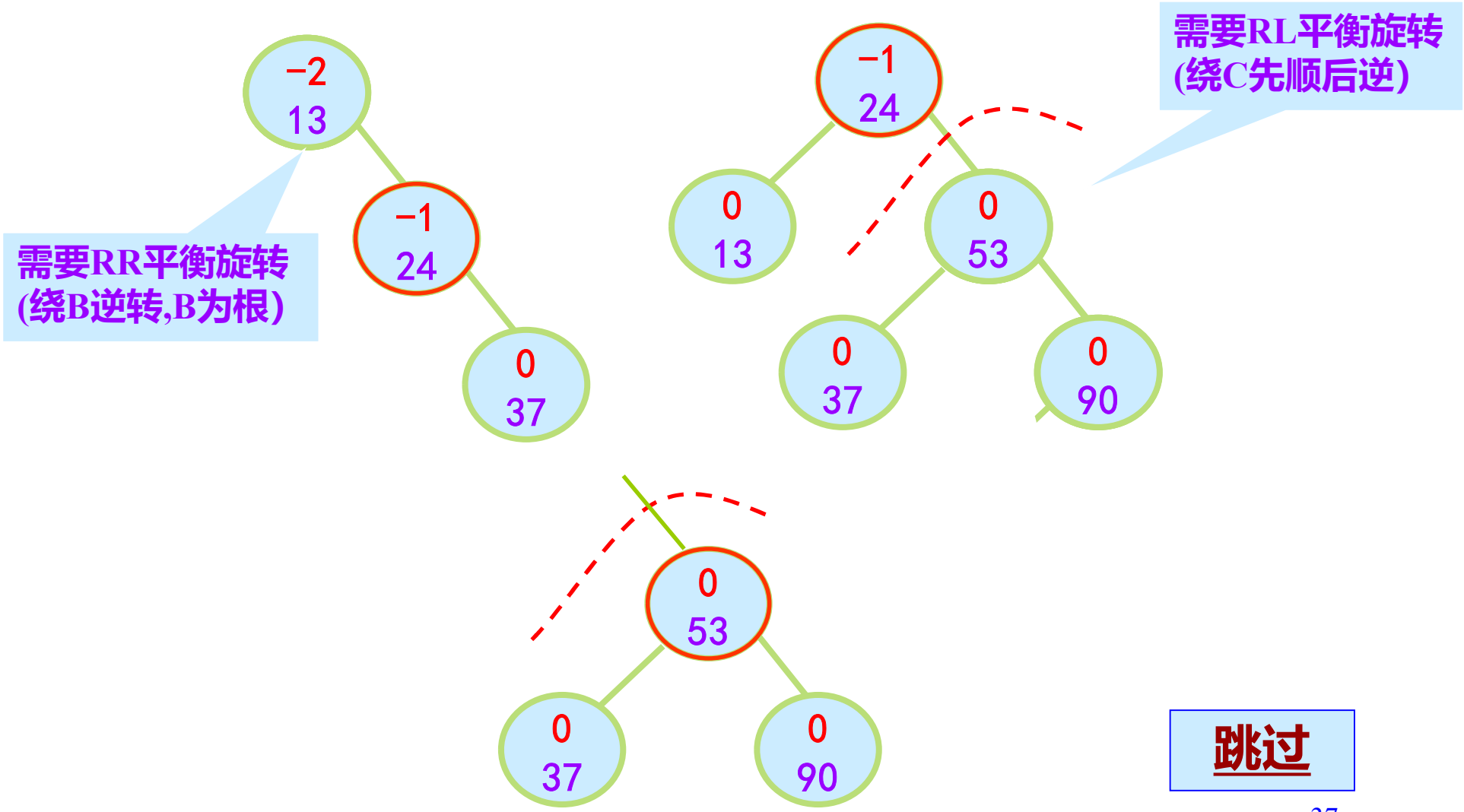
平衡旋转可以归纳为四类：

- LL平衡旋转
- RR平衡旋转
- LR平衡旋转
- RL平衡旋转

这种调整规则可以保证二叉排序树的次序不变

例：请将下面序列构成一棵平衡二叉排序树：

（13，24，37，90，53）

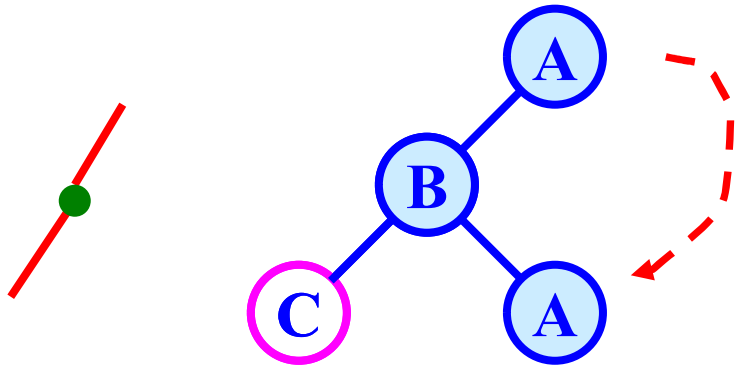


跳过

1）LL平衡旋转：

若在A的左子树的左子树上插入结点，使A的平衡因子从1增加至2，需要进行一次顺时针旋转。

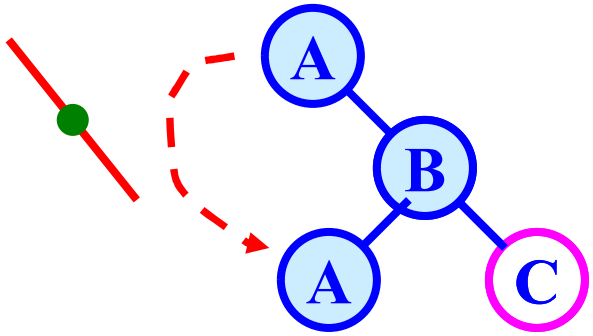
以B为旋转轴



2）RR平衡旋转：

若在A的右子树的右子树上插入结点，使A的平衡因子从-1增加至-2，需要进行一次逆时针旋转。

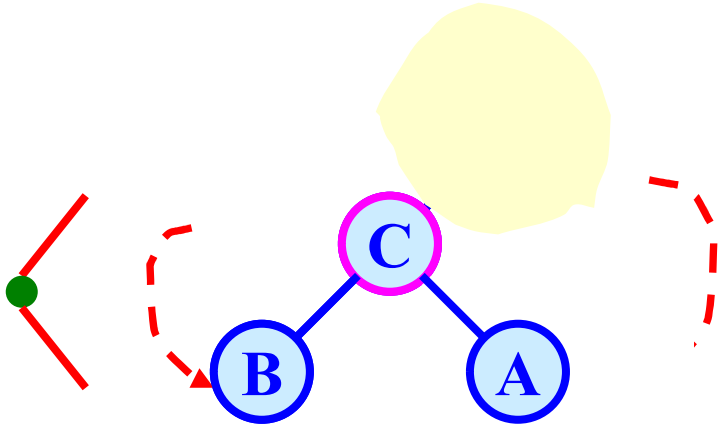
以B为旋转轴



3）LR平衡旋转：

若在A的左子树的右子树上插入结点，使A的平衡因子从1增加至2，需要先进行逆时针旋转，再顺时针旋转。

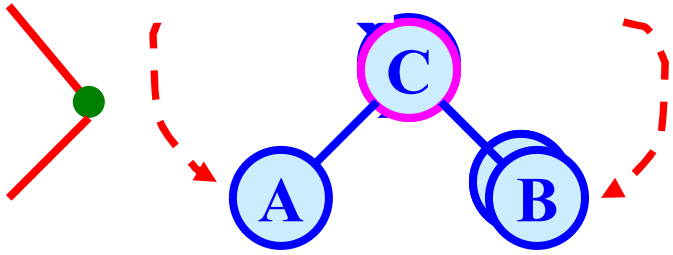
以插入的结点C为旋转轴



4）RL平衡旋转：

若在A的右子树的左子树上插入结点，使A的平衡因子从-1增加至-2，需要先进行顺时针旋转，再逆时针旋转。

以插入的结点C为旋转轴



这种调整规则可以保证二叉排序树的次序不变